Wyatt Habinski
400338858
March 12, 2023

# The Relational Algebra:

COMPSCI 2DB3

## Part 1: The Requested Queries

The solution for each question will begin with the question itself, followed by a small rationale of the relational algebra, and finally the relational algebra query itself.

1) Query: Write a query that returns pairs (cid, fid) in which cid is a comment identifier of a comment on a review that mentions the film with film identifier fid.

We start by joining the table **comment** with **review** on rid as it will relate all comments to their respective article. We then join (**comment** and **review**) with **fmention** on rid again as this will relate the **fmention** rows with their respective reviews. Connecting with **fmention** will also provide the respective film ids.

The relational algebra solution is as follows:

$$\pi_{\text{C.cid,F.fid}} (\varrho_{\text{C}}(\text{comment}) \bowtie_{\texttt{C.rid = R.rid}} \varrho_{\text{R}}(\text{review}) \bowtie_{\texttt{R.rid = F.rid}} \varrho_{\text{F}}(\text{fmention}))$$

2) Query: Write a query that returns film identifiers (fid) of all films that are mentioned in multiple reviews.

We will start with the table **fmention** as it contains a relation between a review and its respective film. In order to acquire films with multiple reviews, we will join ***fmention*** with itself excluding all instances of joining the same (equivalent) rows. Hence we will join on when fmention1.fid is not equal to fmention2.fid. To get the films that have multiple reviews, we further constrain the join to where fmention1.rid is equal to fmention2.rid. Finally we project just the fid.

The relational algebra solution is as follows:

$$\pi_{\text{f1.fid}} (\sigma_{\text{f1.rid = f2.rid}} (\pi_{\text{fid,rid}}(\varrho_{\text{f1}}(\text{fmention}))\bowtie_{\texttt{f1.fid} \neq \texttt{f1.fid}} \pi_{\text{fid,rid}}(\varrho_{\text{f2}}(\text{fmention}))))$$

3) Query: Write a query that returns film identifiers (fid) of all films $f$ that follow the rule of thumb: the film $f$ is assigned to exactly two categories.

This query is equivalent to the difference of two sets: All films with greater or equal to 2 categories, minus All films with greater or equal to 3 categories. The queries for these two sets are similar to the process of question 2 since question 2 can be rephrased as “all films with

greater or equal to 2 reviews. However in this case we will use the table ***fcategory*** as it contains information on both the film and the films category.

The relational algebra solution is as follows:

$$X := \sigma_{\text{f1.category = f2.category}} (\varrho_{\text{f1}}(\text{fcategory}) \bowtie_{\texttt{f1.fid} \neq \texttt{f1.fid}} \varrho_{\text{f2}}(\text{fcategory}))$$

$$Y := \sigma_{\text{f1.category = f2.category = f3.category}} (\varrho_{\text{f1}}(\text{fcategory}) \bowtie_{\texttt{f1.fid} \neq \texttt{f1.fid}} \varrho_{\text{f2}}(\text{fcategory}) \bowtie_{\texttt{f2.fid} \neq \texttt{f3.fid}} \varrho_{\text{f3}}(\text{fcategory}))$$

$$\pi_{\text{f1.fid}} (X \setminus Y)$$

4) Query: Write a query that returns names of categories for which no review mentions a film that is part of that category.

We will use the tables **fcategory** and **fmention** to acquire the category and review information. Since we are looking for any categories with no review mentions of a film in that category, we can left join **fcategory** and **fmention** on equal fids. This will create a new table with all categories, and any respective reviews for that category. Then, any rows with a NULL value for rid will mean that there are no review mentions on a film for that category. We then project the category.

The relational algebra solution is as follows:

$$\pi_{\text{fc.category}} (\sigma_{\text{m.rid = NULL}} (\varrho_{\text{c}}(\text{fcategory}) \ltimes_{\texttt{c.fid = m.fid}} \pi_{\text{fid,rid}}(\varrho_{\text{m}}(\text{fmention})))$$

5) Query: Write a query that returns tuples (rid, uid, fid) such that rid is the review identifier of the latest review $r$ written by the author $a$ with author identifier uid such that $r$ mentions the film $f$ with film identifier fid and $r$ is the latest follow-up review for $f$ written by $a$: there exists another review $s$ written by author $a$ such that both $r$ and $s$ mention the film $f$ and $s$ was written before $r$ . Note that we are only looking for the latest follow-up review by author $a$ for $f$ : there must not be any other review written after review $r$ by author $a$ that mentions the film $f$.

We first find the cross produt of tables **review** and **fmention** to get all combinations. We project only the required coloumns of each table however. We will call this relation **X**

We then take the cross product of **X** and **X** again with constraints where X1.uid = x2.uid to find the reviews made by all the same authors. Also adding the constraint that x1.fid = x2.fid which limits it to reviews made by the same authors and regarding the same films. Lastly the make the constraint that x1.rtime < x2.rtime to get all reviews that are not last. We will call this relation **Y**.

Finally, we find the set of **X** - **Y** in order to find all reviews that are by the same author, regarding the same film, and are not not last (double negation implies that the review is the last review). We then project the required rid, uid, and fid.

The relational algebra solution is as follows:

$$X := (\pi_{R.rid,\ R.uid,\ R.rtime}\ \varrho_R(\text{review})) \times (\pi_{M.rid,\ M.fid}\ \varrho_M(\text{fmention}))$$
$$Y := (\sigma_{X1.uid = X2.uid \wedge X1.fid = X2.fid \wedge X1.rtime < X2.rtime}\ (\varrho_{X1}(X) \times \varrho_{X2}(X)))$$
$$\pi_{Y.rid,\ Y.uid,\ Y.fid}\ (X-Y)$$

6) Query: Write a query that returns a list of film identifiers fid of popular films. A film is a popular film if it has been reviewed by all reviewers. We note that the user table does not only include reviewers: most users only comment, and only the users that ever wrote a review (those that are authors in the review table) are considered reviewers.

Since we are trying to find all films that have been reviewed by all possible authors, we join tables **fmention** and **review** on rid. The table **fmention** will provide all films and their respective reviews. Next, we divide by all possible authors. We can get all possible authors by simply projecting the uid from the **review** table since relational algebra deals with sets and there will be no duplicate authors.
By taking the division of these two sets, we will have only the rows with films that have been reviewed by all authors.

The relational algebra solution is as follows:

$$X := (\varrho_F(\text{fmention}) \bowtie_{F.rid\ =\ R.rid} \varrho_R(\text{review}))$$
$$\pi_{F.fid}\ (X \div (\pi_{R2.uid}\ \varrho_{R2}(\text{review})))$$

**Part 2: Efficiency of Queries**

7) To estimate the size of the output we must estimate each size for each step

Review: has 50 reviewers * 20 reviews each = 1000 * 10 years = 10 000 rows

Film: has 700 a year * 10 years = 7000 rows

Mention: each review mentions 2 films, therefore 1000 reviews * 2 films = 2000 * 10 years = 20 000 rows

Review x Film: 10 000 * 7000 = 70 000 000

(Review x Film) x Mention = 70 000 000 * 20 000 = 1 400 000 000 000

For the $\sigma$ constraint we have:

When R.rid = M.rid: This constraint happens ½ of the time after joining the two tables,
When M.fid = F.dif: This constraint happens ¼ of the time after joining the two tables,
Therefore for both to occur, it would be ½ * ¼ = ⅛ of the time after joining the two tables.

Hence, 1 400 000 000 000 / 8 = 175 000 000 000 rows

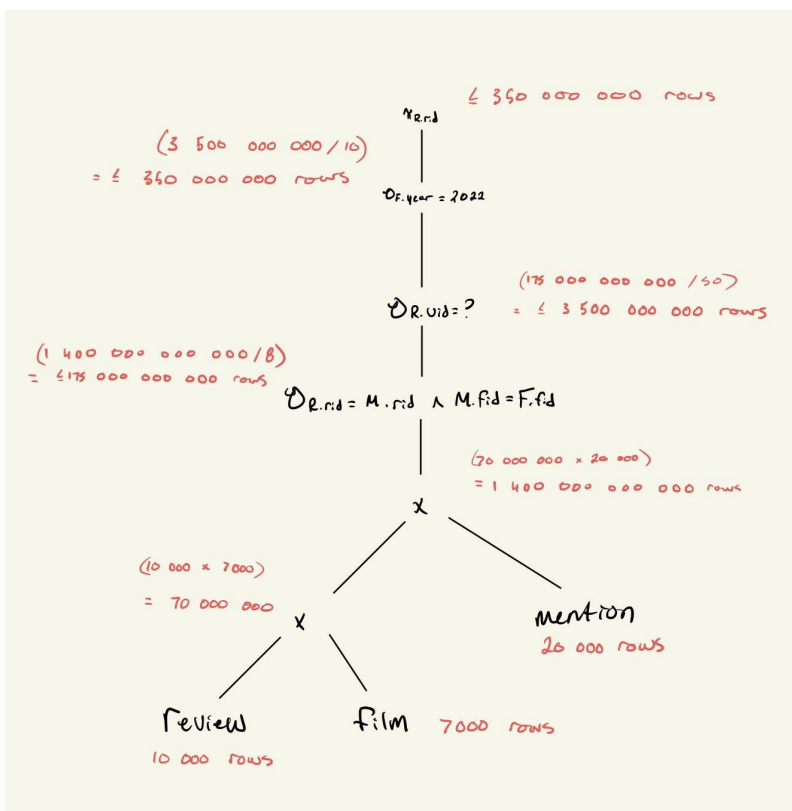
For the next $\sigma$ constraint we have:

When R.uid = ? where ? is a placeholder for the author of the review. Since there are 50 reviewers, this constrains the rows output rows by 1/50. Therefore 175 000 000 000 / 50 = 3 500 000 000 rows

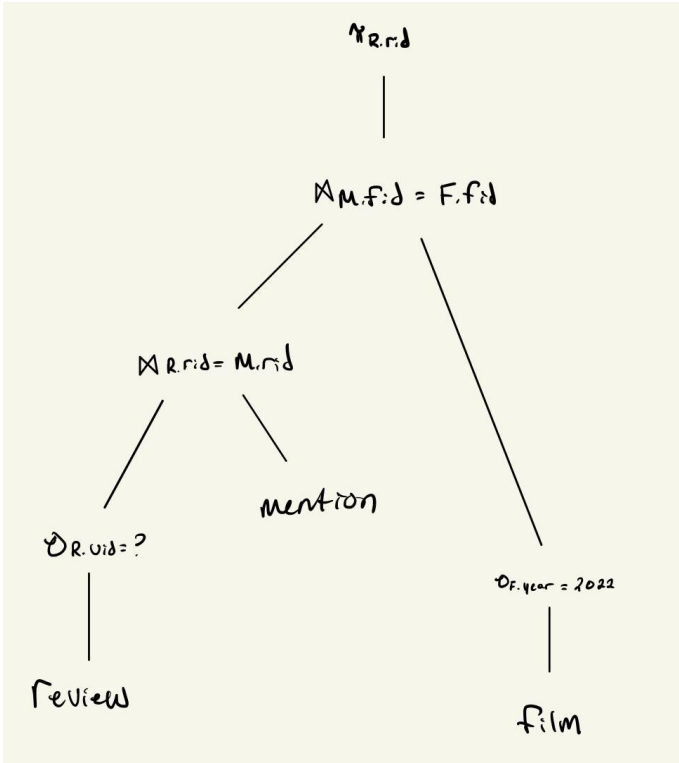
The final $\sigma$ constraint we have:

When F.year = 2022. Since this database holds values for 10 years, this will reduce the number of output rows by a factor of 10. Hence, 3 500 000 000 / 10 = 350 000 000 rows

Finally, the projection $\pi_{\text{R.rid}}$ will remove any duplicate rows which means it will produce at most 350 000 000 rows.



8) First we do not join **review** and **film** since there is no common attribute and would therefore just be the cross product. Instead we join **review** and **mention** on common attributes rid. Then we would join **film** to (**review** and **mention**) by the common attributes fid.
Also, we push down the where constraints as far as possible.
The $\sigma_{R.uid}$ can be called directly from the initial **review** table. The $\sigma_{F.year}$ can be called directly from the **film** table. Then the final two constraints can be implemented during the join stages.
$\sigma_{R.rid = M.mid}$ is used when **review** and **mention** are joined. And the $\sigma_{M.fid = F.fid}$ can be used when **film** is join with (**review** and **mention**).




9) To estimate the size of the output we must estimate each size for each step

Review: still has 10 000 rows

Film: still has 7000 rows

Mention: still has 20 000 rows

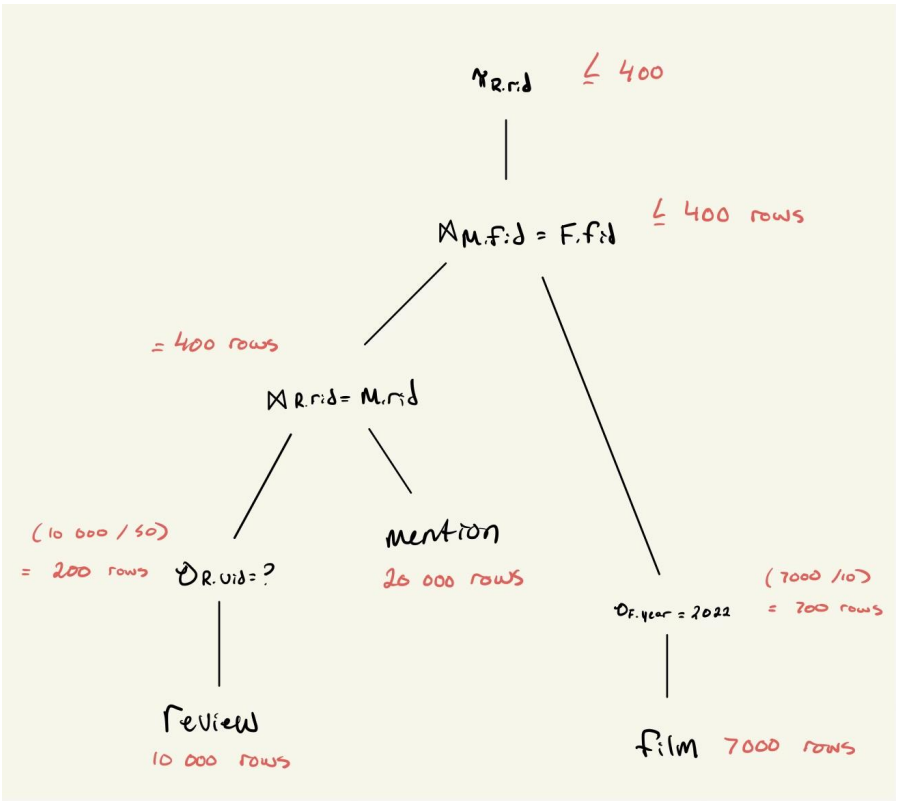
The constraint for review $\sigma_{R.uid = ?}$ is called directly from the review table which instantly reduces the rows by a factor of 50 since there are only 50 authors. Hence 10 000 / 50 = 200 rows

The constraint for films $\sigma_{F.year = 2022}$ is called directly from the film table which instantly reduces the rows by a factor of 10 since there are only 10 years in the database. Hence 7000 / 10 = 700.

Next we have to join **review** and **mention** on R.rid = M.rid. Since every review on average mentions 2  different films, that means there are 2 rows in Mention for every 1 review. Since we have narrowed down the review rows to 200, we will have 200 * 2 = 400 rows.

Next is to combine the previous relation (**review** and **mention)** with **film** on M.fid = F.fid. Since there are only 400 rows in (**review**  and **mention**) but still 700 films, it is possible that every film in a row in (**review**  and **mention**) is unique and therefore we still have at most 400 rows.

Finally, the projection $\pi_{\text{R.rid}}$ will remove any duplicate rows which means it will produce at most 400 rows.



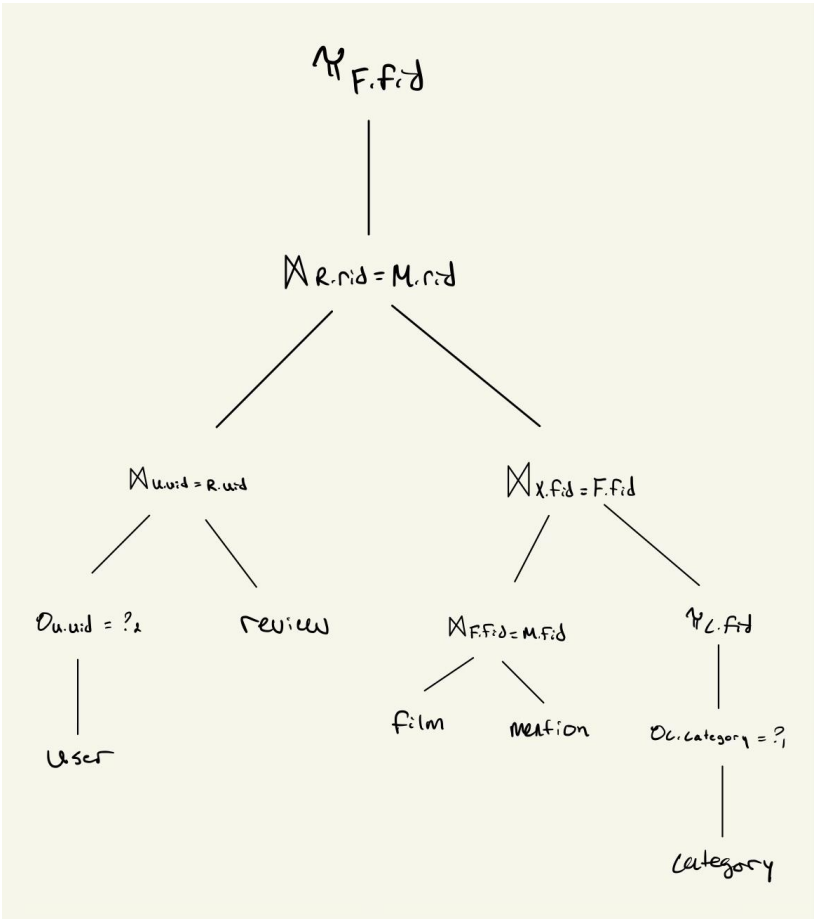

10) An SQL of the original relational algebra query is:

```
SELECT r.rid
FROM REVIEW r, FMENTION m, FILM f
WHERE f.year = 2022
  AND r.uid = ?
  AND r.rid = m.rid
  AND m.fid = f.fid
```

11) The result of translating the SQL into a relational algebra query we get:

$$\pi_{F.fid}\ (\sigma_{U.uid = ?2 \wedge U.uid = R.uid \wedge R.rid = M.rid \wedge M.fid = F.fid = X.fid}$$
$$(\varrho_U(\text{user}) \times \varrho_F(\text{film}) \times \varrho_R(\text{review}) \times \varrho_M(\text{mention}) \times$$
$$\varrho_X(\pi_{C.fid}\ (\sigma_{C.category = ?1}\ \ \varrho_C(\text{category})))))$$

12) We want to be able to push the where clauses as far down as possible. Therefore we will use the $\sigma_{U.uid = ?2}$ constraint directly on the **user** table. We will also use the $\sigma_{C.category = ?1}$ constraint directly on the **category** table.
Next, to avoid large cross products between tables we will join tables that share common attributes. Hence we join **user** and **review** on U.uid = R.uid. We also join tables **film** and **mention** on F.fid = M.fid. Furthermore, we will rename (**film** and **mention)** relation to **X** and join with **category** on X.fid = F.fid. And lastly we join (**user** and **review**) with (**X** and **category**) on R.rid = M.rid.
Finally, we project F.fid




13)
User: The database has 10 000 users and hence the user table will have 10 000 rows

Review: Still has 10 000 rows

Film: Still has 7000 rows

Mention: Still has 20 000 rows

Category: Each film relates to 3 categories. 700 films * 3 categories = 2100 rows

The constraint for $\sigma_{C.category = ?1}$ is finding 1 specific category. Since there is not enough information to deduce how many categories are in the database, I will assume 10. Therefore 2100 / 10 = 210 rows.

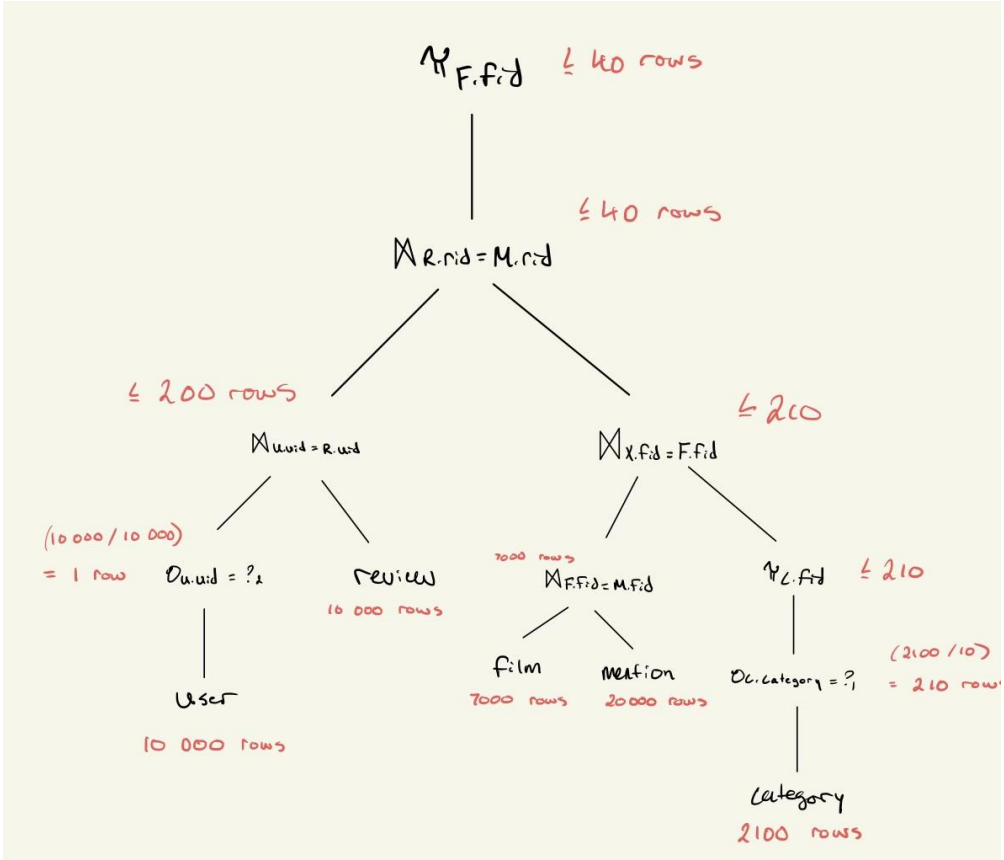
The constraint for $\sigma_{U.uid = ?2}$ is finding 1 specific user out of 10 000 unique users and will therefore output 1 row.

The joining of user and review on uid will output a maximum of 200 rows since each user is expected to put out 20 reviews a year for 10 years. (20 * 10 = 200).

The joining of film and mention on fid will be 7000 rows since every fid in film is unique, and has a respective row in mention.

The joining of (film and mention) and category on fid will also be 210 rows since the category table now has only rows of 1 specific category. And since no film can be in the same category twice, we have 210 distinct films.

The last join of (user and review) with ((film and mention) and category) on rid is essentially asking how many reviews did a single author review that corresponds to movies in a specific category.
Since 1 user outputs 20 reviews a year, and 200 total reviews in 10 years. (200 rows)
And each review corresponds to 2 distinct films. (400 movies and 400 mentions)
But each mention is restricted to 1 of 10 categories, we have 40 rows.
Finally, the projection $\pi_{F.fid}$ will remove any duplicate rows which means it will produce at most 40 rows.


γ F.fid
≤ 40 rows
⋈ R.rid = M.rid
≤ 40 rows
≤ 200 rows
⋈ u.uid = R.uid
⋈ X.fid = F.fid
≤ 210
(10 000 / 10 000)
= 1 row
σ u.uid = ?1
review
10 000 rows
7000 rows
⋈ F.fid = M.fid
γ L.fid
≤ 210
film
7000 rows
mention
20000 rows
σ L.category = ?1
(2100 / 10)
= 210 rows
User
10 000 rows
category
2100 rows